

Product Backlog: ระบบนัดหมายคลินิกสุขภาพในมหาวิทยาลัยและแจ้งเตือนการกินยา

COE67-331 Web Application Development | ทีม 6 คน | Deploy: Vercel | Backend/DB: Supabase

1. User Role (2 แบบ)

Role	รหัส	กลุ่มผู้ใช้งานจริง	ขอบเขตสิทธิ์โดยสรุป
ผู้ใช้ทั่วไป (Patient)	patient	นักศึกษา / บุคลากรที่มาใช้บริการคลินิก	จองคิว, ดู/จัดการข้อมูลของตัวเองเท่านั้น (นัดหมาย, ประวัติรักษา, รายการยา, การแจ้งเตือน)
เจ้าหน้าที่/แอดมิน (Staff)	staff (มี field ย่อย staff_type : nurse / doctor / admin)	พยาบาล, แพทย์, ผู้ดูแลระบบ	จัดการตารางแพทย์, ดู/บันทึกประวัติคนไข้ที่เกี่ยวข้อง, จัดการผู้ใช้ทั้งหมด (เฉพาะ admin), ดู dashboard ภาพรวม

ทั้งสอง role ผูกกับตาราง `users` (field `role`) และควบคุมสิทธิ์การเข้าถึงข้อมูลจริงผ่าน **Supabase Row Level Security (RLS)** ในทุกตารางที่มีข้อมูลอ่อนไหว (PDPA)

2. User Journey (Step-by-step)

2.1 กลุ่ม Patient (นักศึกษา/บุคลากร)

- เข้าเว็บไซต์ → ระบบเช็ค session; ถ้ายังไม่ล็อกอิน redirect ไปหน้า Login/Register
- สมัครสมาชิก (กรอกชื่อ, อีเมล, รหัสผ่าน, รหัสนักศึกษา/รหัสพนักงาน) → ระบบสร้าง record ใน `users` (role = patient) ผ่าน Supabase Auth
- ล็อกอิน → เข้าสู่ **Patient Dashboard** แสดงนัดหมายที่จะถึง + รายการยาที่ต้องกินวันนี้
- ดูตารางแพทย์ว่าง (ดึงจาก `doctor_schedules` ที่ status = available) → เลือกแผนก/แพทย์/วันเวลาที่ต้องการ
- จองคิว → ระบบตรวจสอบว่า slot ยังว่างจริง (กันจองซ้อน) → สร้าง record ใน `appointments` (status = pending/confirmed)
- รับการแจ้งเตือนยืนยันนัด ผ่านแอป/อีเมล (ระบบสร้าง record ใน `notification_logs`)
- ถึงวันนัด → พบแพทย์ตามคิว (แพทย์เป็นฝั่ง staff บันทึกผลตรวจ)
- ดูประวัติการรักษาของตนเอง (อ่านได้เฉพาะของตัวเองจาก `medical_records`)
- รับรายการยาที่ต้องแจ้งเตือน (จากแพทย์สั่ง หรือคนไข้เพิ่มเองใน `medication_reminders`) พร้อมตั้งเวลา/ความถี่
- รับแจ้งเตือนกินยา ตามเวลาจริงผ่านแอป/อีเมล (ระบบ scheduler ยิง notification)
- จัดการนัดหมาย/รายการยาของตน (แก้ไข, ยกเลิก, ทำเครื่องหมายว่ากินยาแล้ว)
- ออกจากระบบ (Logout)

2.2 กลุ่ม Staff/Admin (พยาบาล/แพทย์/แอดมิน)

- เข้าเว็บไซต์ → ล็อกอิน ด้วยบัญชี staff → ระบบตรวจ role และพาเข้า **Staff Dashboard**
- จัดการตารางแพทย์: เพิ่ม/แก้ไข/ลบ slot เวลาว่างของแพทย์แต่ละคนในแต่ละวัน
- ดูรายการนัดหมายที่เข้ามาใหม่ → ยืนยัน/ปฏิเสธ/เลื่อนนัด
- เมื่อคนไข้มาถึงคลินิก: เปิดข้อมูลนัดหมาย → บันทึกประวัติการรักษา (การวินิจฉัย, การรักษา, หมายเหตุ)
- ส่งจ่ายยา/ตั้งรายการแจ้งเตือนยา ให้คนไข้ (เชื่อมกับประวัติการรักษา)
- ดู Dashboard สรุปภาพรวม (จำนวนนัดหมายวันนี้, สถิติการแจ้งเตือนที่ส่งสำเร็จ/ล้มเหลว, จำนวนผู้ใช้งาน) — เฉพาะ admin
- จัดการบัญชีผู้ใช้ (เฉพาะ admin): ดูรายชื่อผู้ใช้ทั้งหมด, เปลี่ยน role, ระบุ/ลบบัญชี
- ตั้งค่าช่องทางแจ้งเตือนของระบบ (แอป/อีเมล) และตรวจสอบ log การส่ง
- ออกจากระบบ (Logout)

3. Features Specification, User Stories & Acceptance Criteria

รูปแบบ User Story: "ในฐานะ [ผู้], ฉันต้องการ [ทำอะไร], เพื่อที่จะ [ได้ประโยชน์อะไร]" รหัส: `US-Fx.y` = User Story ของฟีเจอร์ x ข้อ y | `AC-Fx.y` = Acceptance Criteria ของ US เดียวกัน

◆ Feature 1: Authentication & Profile Management (สมาชิก 1)

ตารางหลัก: `users` | หน้าที่: สมัคร/ล็อกอิน/จัดการโปรไฟล์/จัดการสิทธิ์ผู้ใช้

CRUD	รายละเอียด
Create	สมัครสมาชิกใหม่ (patient/staff)
Read	ดูโปรไฟล์ตัวเอง / admin ดูรายชื่อผู้ใช้ทั้งหมด
Update	แก้ไขข้อมูลส่วนตัว / admin เปลี่ยน role
Delete	ปิดใช้งานบัญชี (soft delete) / admin ลบผู้ใช้
Auth	Supabase Auth (email+password), session/JWT, RLS แยกสิทธิ์ patient/staff

User Stories

- US-F1.1** (Functional): ในฐานะ **ผู้ใช้ใหม่**, ฉันต้องการ **สมัครสมาชิกด้วยอีเมลและกำหนดรหัสผ่าน**, เพื่อที่จะ **มีบัญชีสำหรับใช้งานระบบนัดหมายคลินิก**
- US-F1.2** (Functional): ในฐานะ **ผู้ใช้ที่ล็อกอินแล้ว**, ฉันต้องการ **แก้ไขข้อมูลส่วนตัวของฉันได้**, เพื่อที่จะ **ให้ข้อมูลติดต่อ/ข้อมูลนัดหมายถูกต้องอยู่เสมอ**
- US-F1.3** (Functional): ในฐานะ **แอดมิน**, ฉันต้องการ **ดูรายชื่อผู้ใช้ทั้งหมดและปรับสิทธิ์ (role) ได้**, เพื่อที่จะ **บริหารจัดการบัญชีเจ้าหน้าที่และคนไข้ในระบบ**
- US-F1.4** (Non-Functional): ในฐานะ **ผู้ดูแลระบบ**, ฉันต้องการ **ให้รหัสผ่านผู้ใช้ถูกเข้ารหัสและ session หมดอายุอัตโนมัติเมื่อไม่ใช้งาน**, เพื่อที่จะ **ป้องกันข้อมูลบัญชีผู้ใช้จากการเข้าถึงโดยไม่ได้รับอนุญาต**

Acceptance Criteria

ID	User Story	Acceptance Criteria
AC-F1.1	US-F1.1	มี Form สมัครสมาชิก (ชื่อ, อีเมล, รหัสผ่าน, ยืนยันรหัสผ่าน, รหัสนักศึกษา/พนักงาน) ตรวจสอบ validation ฝั่ง client (email format, password ≥ 8 ตัวอักษร) ก่อนส่ง เมื่อกดปุ่ม "สมัครสมาชิก" ต้องมี Loading state บนปุ่ม และ disable ปุ่มระหว่างส่งข้อมูล บันทึกข้อมูลจริงลงตาราง <code>users</code> ผ่าน Supabase Auth signUp API พร้อม role เริ่มต้น = patient หากอีเมลซ้ำ ต้องแสดง Error message ชัดเจน ("อีเมลนี้ถูกใช้แล้ว") ไม่ใช่ error ทั่วไป
AC-F1.2	US-F1.2	หน้าโปรไฟล์ดึงข้อมูลจริงจาก <code>users</code> มาแสดงใน Form ที่แก้ไขได้ (pre-filled) กดปุ่ม "บันทึก" ต้องมี Loading state และแสดง Toast/Alert ยืนยันเมื่อบันทึกสำเร็จ ตั้ง RLS Policy บน <code>users</code> : <code>UPDATE</code> อนุญาตเฉพาะ <code>auth.uid() = id</code> (แก้ไขได้เฉพาะของตัวเอง) หากบันทึกไม่สำเร็จ (เช่น เน็ตหลุด) ต้องแสดง Error state พร้อมปุ่ม "ลองอีกครั้ง"
AC-F1.3	US-F1.3	หน้า Admin แสดงตาราง (table) รายชื่อผู้ใช้ทั้งหมด ดึงจาก <code>users</code> จริง พร้อม pagination/search มี Dropdown เปลี่ยน role ต่อแถว และปุ่ม "ระงับบัญชี" ที่ต้องมี Confirm Dialog ก่อนดำเนินการ ตั้ง RLS Policy: <code>SELECT/UPDATE</code> ทั้งตารางอนุญาตเฉพาะ role = admin เท่านั้น แสดง Loading skeleton ระหว่างดึงข้อมูล และ Empty state หากไม่มีผู้ใช้ตรงเงื่อนไขค้นหา
AC-F1.4	US-F1.4	รหัสผ่านไม่ถูกเก็บเป็น plain text (ใช้ Supabase Auth hashing ในตัว) Session token หมดอายุตามค่าที่ตั้งไว้ (เช่น 24 ชม.) และ auto-redirect ไป login เมื่อ token invalid ทุก API call ที่แก้ไขข้อมูลผู้ใช้ต้องตรวจสอบ JWT ก่อนเสมอ (ฝั่ง Supabase RLS + client guard)

◆ Feature 2: Doctor Schedule Management (สมาชิก 2)

ตารางหลัก: `doctor_schedules` | หน้าที่: เจ้าหน้าที่จัดการตารางเวลาวางของแพทย์

CRUD	รายละเอียด
Create	staff เพิ่ม slot เวลาวางของแพทย์ (แผนก, วัน, เวลาเริ่ม-สิ้นสุด)
Read	patient/staff ดูตารางแพทย์ว่างทั้งหมด
Update	staff แก้ไขเวลา/สถานะ slot
Delete	staff ลบ slot ที่ยังไม่ถูกจอง
Auth	เฉพาะ role = staff ที่มีสิทธิ์ C/U/D, patient อ่านได้อย่างเดียว

User Stories

- US-F2.1** (Functional): ในฐานะ **เจ้าหน้าที่คลินิก**, ฉันต้องการ **เพิ่มตารางเวลาวางของแพทย์แต่ละคน**, เพื่อที่จะ **ให้คนไข้เห็นช่วงเวลาที่จะจองได้จริง**
- US-F2.2** (Functional): ในฐานะ **คนไข้**, ฉันต้องการ **ดูตารางแพทย์ว่างตามแผนก/วันที่**, เพื่อที่จะ **เลือกเวลานัดหมายที่สะดวก**
- US-F2.3** (Functional): ในฐานะ **เจ้าหน้าที่คลินิก**, ฉันต้องการ **แก้ไขหรือลบ slot ที่ตั้งผิด**, เพื่อที่จะ **ให้ตารางแพทย์ถูกต้องอยู่เสมอ**
- US-F2.4** (Non-Functional): ในฐานะ **ระบบ**, ฉันต้องการ **ป้องกันไม่ให้สร้าง slot เวลาที่ซ้อนทับกันของแพทย์คนเดียวกัน**, เพื่อที่จะ **รักษาความถูกต้องของ**

ข้อมูลตารางเวลา

Acceptance Criteria

ID	User Story	Acceptance Criteria
AC-F2.1	US-F2.1	มี Form insert (เลือกแพทย์/แผนก, วันที่, เวลาเริ่ม, เวลาสิ้นสุด, ระยะเวลาต่อคิว) Validate: เวลาสิ้นสุดต้องมากกว่าเวลาเริ่ม, ไม่สามารถเลือกวันที่ย้อนหลังได้ บันทึกจริงลง <code>doctor_schedules</code> พร้อม Loading state ตอนกด submit RLS Policy: <code>INSERT</code> อนุญาตเฉพาะ role = staff
AC-F2.2	US-F2.2	หน้ารายการตารางแพทย์ดึงข้อมูลจริงจาก <code>doctor_schedules</code> filter ตามแผนก/วันที่ที่เลือก แสดงเฉพาะ slot ที่ status = available (ซ่อน slot ที่ถูกจองเต็มแล้ว) มี Loading skeleton ระหว่างโหลด และ Empty state ถ้าวันนั้นไม่มี slot ว่าง
AC-F2.3	US-F2.3	แต่ละแถวมีปุ่ม Edit (เปิด Form แก้ไข pre-filled) และปุ่ม Delete ปุ่ม Delete ต้องมี Confirm Dialog ("ยืนยันการลบตารางนี้?") ก่อนลบจริง ถ้า slot มีการจองแล้ว (ผูกกับ <code>appointments</code>) ต้อง disable ปุ่ม Delete และแสดงข้อความแจ้งเตือน RLS Policy: <code>UPDATE/DELETE</code> อนุญาตเฉพาะ role = staff
AC-F2.4	US-F2.4	ก่อน insert/update ตรวจสอบ (server-side หรือ DB constraint) ว่าไม่มี slot ของแพทย์คนเดียวกันที่เวลาซ้อนทับ ถ้าซ้อนทับ ต้องแสดง Error message เฉพาะเจาะจง ("ช่วงเวลานี้ซ้อนกับตารางที่มีอยู่แล้ว") ไม่ให้บันทึก

◆ Feature 3: Appointment Booking (สมาชิก 3)

ตารางหลัก: `appointments` | หน้าที่: จองคิว/นัดหมาย พร้อมป้องกันการจองซ้อน

CRUD	รายละเอียด
Create	patient จองคิวจาก slot ว่าง
Read	patient ดูนัดหมายของตัวเอง / staff ดูนัดหมายทั้งหมด
Update	เลื่อนนัด/ยืนยัน/ยกเลิกนัด (เปลี่ยน status)
Delete	admin ลบ record นัดหมาย (rare case)
Auth	RLS: patient เห็นเฉพาะนัดของตัวเอง, staff เห็นทั้งหมด

User Stories

- US-F3.1 (Functional): ในฐานะ **คนไข้**, ฉันต้องการ **จองคิวนัดหมายกับแพทย์ในเวลาที่ย่าง**, เพื่อที่จะ **ได้รับการตรวจรักษาตามเวลาที่สะดวก**
- US-F3.2 (Functional): ในฐานะ **คนไข้**, ฉันต้องการ **ดูและยกเลิก/เลื่อนนัดหมายของตัวเอง**, เพื่อที่จะ **บริหารเวลาของตัวเองได้เมื่อมีเหตุขัดข้อง**
- US-F3.3 (Functional): ในฐานะ **เจ้าหน้าที่คลินิก**, ฉันต้องการ **ดูรายการนัดหมายทั้งหมดที่เข้ามาในแต่ละวัน**, เพื่อที่จะ **เตรียมความพร้อมและจัดคิวตรวจได้ถูกต้อง**
- US-F3.4 (Non-Functional): ในฐานะ **ระบบ**, ฉันต้องการ **ตรวจสอบและป้องกันการจองคิวซ้อนกันในเวลาเดียวกัน (race condition)**, เพื่อที่จะ **ไม่ให้เกิดการจองซ้ำในช่วงเวลาเดียวกันของแพทย์คนเดียว**

Acceptance Criteria

ID	User Story	Acceptance Criteria
AC-F3.1	US-F3.1	หน้าจองคิวมี Form/UI เลือก slot จาก <code>doctor_schedules</code> (Calendar/Date-Time Picker component) แล้วกด "ยืนยันการจอง" ปุ่มยืนยันมี Loading state ระหว่างตรวจสอบและบันทึก บันทึกจริงลง <code>appointments</code> (status = pending) พร้อมอัปเดตสถานะ slot ที่เกี่ยวข้อง หากจองสำเร็จ แสดง Success message/redirect ไปหน้า "นัดหมายของฉัน"
AC-F3.2	US-F3.2	หน้า "นัดหมายของฉัน" ดึงข้อมูลจริงจาก <code>appointments</code> filter <code>patient_id = auth.uid()</code> แต่ละรายการมีปุ่ม "ยกเลิกนัด" พร้อม Confirm Dialog และปุ่ม "เลื่อนนัด" ที่เปิด Form เลือกวันเวลาใหม่ RLS Policy: <code>SELECT/UPDATE</code> บน <code>appointments</code> อนุญาตเฉพาะเจ้าของ (<code>patient_id = auth.uid()</code>) มี Loading state และ Empty state ("ยังไม่มีนัดหมาย")
AC-F3.3	US-F3.3	หน้า Staff แสดงตารางนัดหมายทั้งหมด (real-time หรือ refresh) filter ตามวันที่/แพทย์/สถานะ มีปุ่มเปลี่ยนสถานะ (ยืนยัน/ปฏิเสธ) ต่อแถว พร้อม Loading state ตอนกด RLS Policy: <code>SELECT</code> ทั้งตารางอนุญาตเฉพาะ role = staff
AC-F3.4	US-F3.4	เมื่อกด "ยืนยันการจอง" ระบบต้องเช็คสถานะ slot ณ เวลานั้นอีกครั้งก่อน insert จริง (เช่นใช้ DB transaction หรือ unique constraint บน <code>schedule_id</code>) หากมีคนอื่นจองไปแล้วในเวลาใกล้เคียงกัน ต้องแสดง Error message ("ช่วงเวลานี้เพิ่งถูกจองไปแล้ว กรุณาเลือกใหม่") และรีเฟรชรายการ slot อัตโนมัติ

◆ Feature 4: Medical Records Management (สมาชิก 4)

ตารางหลัก: `medical_records` | หน้าที่: บันทึก/ดูประวัติการรักษา (ข้อมูลอ่อนไหวตาม PDPA)

CRUD	รายละเอียด
Create	staff/แพทย์บันทึกผลตรวจหลังพบคนไข้
Read	patient ดูประวัติของตัวเอง / staff ดูของคนไข้ที่เกี่ยวข้อง
Update	แพทย์แก้ไขบันทึกการวินิจฉัย/การรักษา
Delete	admin ลบบันทึก (กรณีพิเศษ, ต้องมี audit)
Auth	RLS เข้มงวดสุด: patient เห็นเฉพาะของตัวเอง, staff เห็นเฉพาะที่เกี่ยวข้อง (PDPA)

User Stories

- US-F4.1 (Functional): ในฐานะ แพทย์, ฉันต้องการ บันทึกผลการวินิจฉัยและการรักษาหลังตรวจคนไข้แต่ละครั้ง, เพื่อที่จะ มีประวัติการรักษาไว้ใช้อ้างอิงครั้งถัดไป
- US-F4.2 (Functional): ในฐานะ คนไข้, ฉันต้องการ ดูประวัติการรักษาของตัวเองย้อนหลังได้, เพื่อที่จะ ติดตามอาการและการรักษาที่ผ่านมาของตนเอง
- US-F4.3 (Functional): ในฐานะ แพทย์, ฉันต้องการ แก้ไขบันทึกประวัติที่กรอกผิดพลาด, เพื่อที่จะ ให้ข้อมูลการรักษาถูกต้องแม่นยำ
- US-F4.4 (Non-Functional): ในฐานะ ผู้ดูแลระบบ, ฉันต้องการ จำกัดสิทธิ์การเข้าถึงข้อมูลประวัติรักษาอย่างเข้มงวดตาม PDPA, เพื่อที่จะ ป้องกันไม่ให้บุคคลที่ไม่เกี่ยวข้องเข้าถึงข้อมูลสุขภาพของผู้อื่น

Acceptance Criteria

ID	User Story	Acceptance Criteria
AC-F4.1	US-F4.1	มี Form insert (เลือก appointment ที่เกี่ยวข้อง, ช่องกรอกการวินิจฉัย, การรักษา, หมายเหตุ) Validate: ฟیلด์การวินิจฉัยห้ามว่าง ก่อนบันทึก ปุ่มบันทึกมี Loading state และ Success message เมื่อบันทึกลง <code>medical_records</code> สำเร็จ RLS Policy: <code>INSERT</code> อนุญาตเฉพาะ role = staff (staff_type = doctor/nurse)
AC-F4.2	US-F4.2	หน้า "ประวัติการรักษา" ดึงข้อมูลจริงจาก <code>medical_records</code> filter <code>patient_id = auth.uid()</code> เท่านั้น แสดงผลแบบ timeline/list เรียงตามวันที่ล่าสุดก่อน พร้อม Loading skeleton RLS Policy: <code>SELECT</code> อนุญาตเฉพาะเจ้าของ record หรือ staff ที่เกี่ยวข้อง — ห้าม patient คนอื่นเข้าถึงได้เด็ดขาด Empty state หากยังไม่มีประวัติ
AC-F4.3	US-F4.3	แต่ละ record มีปุ่ม "แก้ไข" เปิด Form pre-filled ข้อมูลเดิม บันทึกการแก้ไขต้องมี Confirm Dialog ก่อนบันทึกทับ (เพราะเป็นข้อมูลสำคัญ) RLS Policy: <code>UPDATE</code> อนุญาตเฉพาะ staff ผู้บันทึก record นั้น หรือ admin
AC-F4.4	US-F4.4	ตั้ง RLS Policy บน <code>medical_records</code> แยกตาม role อย่างชัดเจน (patient: own only, staff: assigned only, admin: all) ทดสอบว่าเมื่อ patient A พยายามเรียก API ดูข้อมูลของ patient B จะได้ผลลัพธ์ว่างเปล่า/403 เสมอ Sensitive field (เช่น การวินิจฉัย) ต้องไม่ถูก log ไว้ใน client-side console/analytics

◆ Feature 5: Medication Reminder & Scheduler (สมาชิก 5)

ตารางหลัก: `medication_reminders` | หน้าที่: ตั้งเวลาแจ้งเตือนการกินยารายบุคคล + scheduler ฝั่ง server

CRUD	รายละเอียด
Create	staff (จากการวินิจฉัย) หรือ patient สร้างรายการยา/เวลาแจ้งเตือน
Read	patient ดูตารางยาของตัวเอง / staff ดูของคนไข้ที่เกี่ยวข้อง
Update	แก้ไขเวลา/ความถี่/ขนาดยา, mark ว่ากินแล้ว
Delete	ลบรายการยาที่หยุดใช้แล้ว
Auth	RLS เหมือน F4 (ข้อมูลผูกกับสุขภาพส่วนบุคคล)

User Stories

- US-F5.1 (Functional): ในฐานะ แพทย์, ฉันต้องการ ตั้งรายการยาและเวลาที่คนไข้ต้องกิน, เพื่อที่จะ ให้คนไข้กินยาตรงเวลาตามที่สั่ง
- US-F5.2 (Functional): ในฐานะ คนไข้, ฉันต้องการ ดูตารางยาของตัวเองและทำเครื่องหมายเมื่อกินยาแล้ว, เพื่อที่จะ ติดตามการกินยาของตัวเองได้อย่างถูกต้อง
- US-F5.3 (Functional): ในฐานะ คนไข้, ฉันต้องการ แก้ไขหรือลบรายการยาที่หยุดใช้แล้ว, เพื่อที่จะ ไม่ให้ระบบแจ้งเตือนยาที่ไม่ต้องกินแล้ว

- **US-F5.4** (Non-Functional): ในฐานะ ระบบ, ฉันต้องการ มี scheduler ที่ตรวจสอบเวลาแจ้งเตือนของทุกคนแบบ real-time (background job), เพื่อที่จะส่งการแจ้งเตือนกันยาได้ตรงเวลาริงของแต่ละคน

Acceptance Criteria

ID	User Story	Acceptance Criteria
AC-F5.1	US-F5.1	มี Form insert (ชื่อยา, ขนาด/dosage, ความถี่ เช่น วันละ 3 ครั้ง, เวลาที่ต้องกิน, ผูกกับ <code>medical_record_id</code>) Validate: เวลาที่กรอกต้องอยู่ในรูปแบบเวลาที่ถูกต้อง ห้ามเป็นค่าว่าง ปุ่มบันทึกมี Loading state, บันทึกจริงลง <code>medication_reminders</code>
AC-F5.2	US-F5.2	หน้า "ตารางยาของฉัน" ดึงข้อมูลจริงจาก <code>medication_reminders</code> filter <code>patient_id = auth.uid()</code> แสดงเป็นรายการต่อวัน มี Checkbox/ปุ่ม "กินยาแล้ว" ที่อัปเดตสถานะทันที (optimistic update + rollback ถ้า error) RLS Policy: <code>SELECT/UPDATE</code> เฉพาะเจ้าของ record
AC-F5.3	US-F5.3	ปุ่ม "แก้ไข" เปิด Form pre-filled, ปุ่ม "ลบ" ต้องมี Confirm Dialog เมื่อแก้ไข/ลบสำเร็จ ต้อง sync สถานะกับตาราง <code>notification_logs</code> (ยกเลิกการแจ้งเตือนที่ยังไม่ส่ง)
AC-F5.4	US-F5.4	มี Scheduler ฝั่ง server (เช่น Supabase Edge Function + Cron หรือ pg_cron) ที่รันตรวจสอบ <code>reminder_time</code> เทียบกับเวลาปัจจุบันทุก 1-5 นาที เมื่อถึงเวลาแจ้งเตือน ระบบต้อง insert record ใหม่ใน <code>notification_logs</code> โดยอัตโนมัติ (ไม่ต้องมี user action) ทดสอบ edge case: ตั้งเวลาที่ผ่านไปแล้ว (อดีต) ต้อง validate error ไม่ให้บันทึกได้

◆ Feature 6: Notification System & Admin Dashboard (สมาชิก 6)

ตารางหลัก: `notification_logs` | หน้าที่: ส่ง/บันทึก log การแจ้งเตือนผ่านแอป/อีเมล + สรุปภาพรวมสำหรับแอดมิน

CRUD	รายละเอียด
Create	ระบบสร้าง log อัตโนมัติเมื่อส่งแจ้งเตือน
Read	patient ดูประวัติแจ้งเตือนตัวเอง / admin ดู dashboard รวมทุกคน
Update	ผู้ใช้ตั้งค่าช่องทางแจ้งเตือน (app/email/ทั้งสอง)
Delete	admin ลบ log เก่า (data retention)
Auth	RLS: patient เห็นเฉพาะ log ของตัวเอง, dashboard สรุปเฉพาะ admin

User Stories

- **US-F6.1** (Functional): ในฐานะ คนไข้, ฉันต้องการ เลือกช่องทางรับการแจ้งเตือน (แอป/อีเมล), เพื่อที่จะ ได้รับข่าวสารในช่องทางที่สะดวกที่สุด
- **US-F6.2** (Functional): ในฐานะ คนไข้, ฉันต้องการ ดูประวัติการแจ้งเตือนที่เคยได้รับ, เพื่อที่จะ ตรวจสอบย้อนหลังว่าไม่พลาดนัดหมายหรือการกินยา
- **US-F6.3** (Functional): ในฐานะ แอดมิน, ฉันต้องการ ดู Dashboard สรุปสถิติการใช้งานระบบ, เพื่อที่จะ ติดตามภาพรวมและประสิทธิภาพของระบบแจ้งเตือน
- **US-F6.4** (Non-Functional): ในฐานะ ระบบ, ฉันต้องการ บันทึกสถานะการส่งแจ้งเตือนทุกครั้ง (สำเร็จ/ล้มเหลว), เพื่อที่จะ ตรวจสอบและแก้ไขปัญหาการแจ้งเตือนที่ผิดพลาดได้

Acceptance Criteria

ID	User Story	Acceptance Criteria
AC-F6.1	US-F6.1	หน้า "ตั้งค่าแจ้งเตือน" มี Toggle/Checkbox เลือกช่องทาง (App/Email) พร้อมปุ่มบันทึก + Loading state Validate รูปแบบอีเมลก่อนบันทึก หากรูปแบบผิดต้องแสดง Error message บันทึกค่าจริงผูกกับ <code>users</code> หรือ field การตั้งค่าที่เกี่ยวข้อง
AC-F6.2	US-F6.2	หน้า "ประวัติการแจ้งเตือน" ดึงข้อมูลจริงจาก <code>notification_logs</code> filter ตาม user ที่เกี่ยวข้อง (ผ่าน <code>reminder_id</code> / <code>patient_id</code>) เรียงตามเวลาล่าสุด แสดง badge สถานะ (ส่งสำเร็จ/ล้มเหลว) พร้อม Loading skeleton และ Empty state
AC-F6.3	US-F6.3	หน้า Admin Dashboard แสดงกราฟ/การ์ดสรุป (จำนวนนัดหมายวันนี้, จำนวนการแจ้งเตือนที่ส่งสำเร็จ/ล้มเหลว, จำนวนผู้ใช้งานทั้งหมด) ดึงจากข้อมูลจริงในหลายตาราง มี Loading state ระหว่างคำนวณสรุปผล และปุ่ม Refresh ข้อมูล RLS Policy: เข้าถึง endpoint/ตารางสรุปได้เฉพาะ role = admin
AC-F6.4	US-F6.4	ทุกครั้งที่ scheduler (F5) พยายามส่งแจ้งเตือน ต้อง insert record ใน <code>notification_logs</code> พร้อม field <code>status</code> (success/failed) และ <code>error_message</code> ถ้ามี Admin สามารถลบ log ที่เก่ากว่ากำหนด (เช่น > 6 เดือน) ผ่านปุ่มที่มี Confirm Dialog

4. สรุปสำหรับนำไปใช้ต่อ (ER Diagram / Use Case Diagram)

4.1 ตารางฐานข้อมูล (6 ตาราง) และความสัมพันธ์

ตาราง	Primary Key	Foreign Key เชื่อมไปยัง
users	id	–
doctor_schedules	id	–
appointments	id	patient_id → users.id, schedule_id → doctor_schedules.id
medical_records	id	appointment_id → appointments.id, patient_id → users.id, recorded_by → users.id
medication_reminders	id	patient_id → users.id, record_id → medical_records.id
notification_logs	id	reminder_id → medication_reminders.id

ความสัมพันธ์แบบง่าย: users (1)–(M) appointments (M)–(1) doctor_schedules appointments (1)–(1) medical_records (1)–(1) medication_reminders (1)–(M) notification_logs

4.2 Actor สำหรับ Use Case Diagram

- **Patient** (นักศึกษา/บุคลากร) → สมัคร/ล็อกอิน, จองคิว, ยกเลิก/เลื่อนนัด, ดูประวัติรักษา, จัดการตารางยา, ตั้งค่าแจ้งเตือน
- **Staff (nurse/doctor)** → จัดการตารางแพทย์, ยืนยันนัดหมาย, บันทึก/แก้ไขประวัติรักษา, สั่งจ่ายยา
- **Admin** → จัดการผู้ใช้ทั้งหมด, ดู Dashboard สรุป, จัดการ log การแจ้งเตือน

4.3 การมอบหมายวาดภาพ

- **ER Diagram:** มอบให้สมาชิกคนที่ 3 (เจ้าของตาราง `appointments` ซึ่งเป็นจุดเชื่อมกลาง)
- **Use Case Diagram:** มอบให้สมาชิกคนที่ 1 (เจ้าของ Authentication ที่เห็นภาพรวม role ชัดเจนที่สุด)

5. Checklist สรุป

- ☒ User Role 2 แบบ พร้อมสิทธิ์ชัดเจน
- ☒ User Journey ครบทั้ง 2 กลุ่ม (Patient / Staff)
- ☒ 6 ฟังก์ชัน ครอบคลุม CRUD + Authentication ครบทุกฟังก์ชัน
- ☒ User Story ฟังก์ชัน "ในฐานะ...ฉันต้องการ...เพื่อที่จะ..." ครบ Functional + Non-Functional
- ☒ Acceptance Criteria แบบตาราง (ID / User Story / AC) ลงรายละเอียด Technical + UI + RLS
- ☒ ข้อมูลพร้อมต่อจัดทำ ER Diagram และ Use Case Diagram